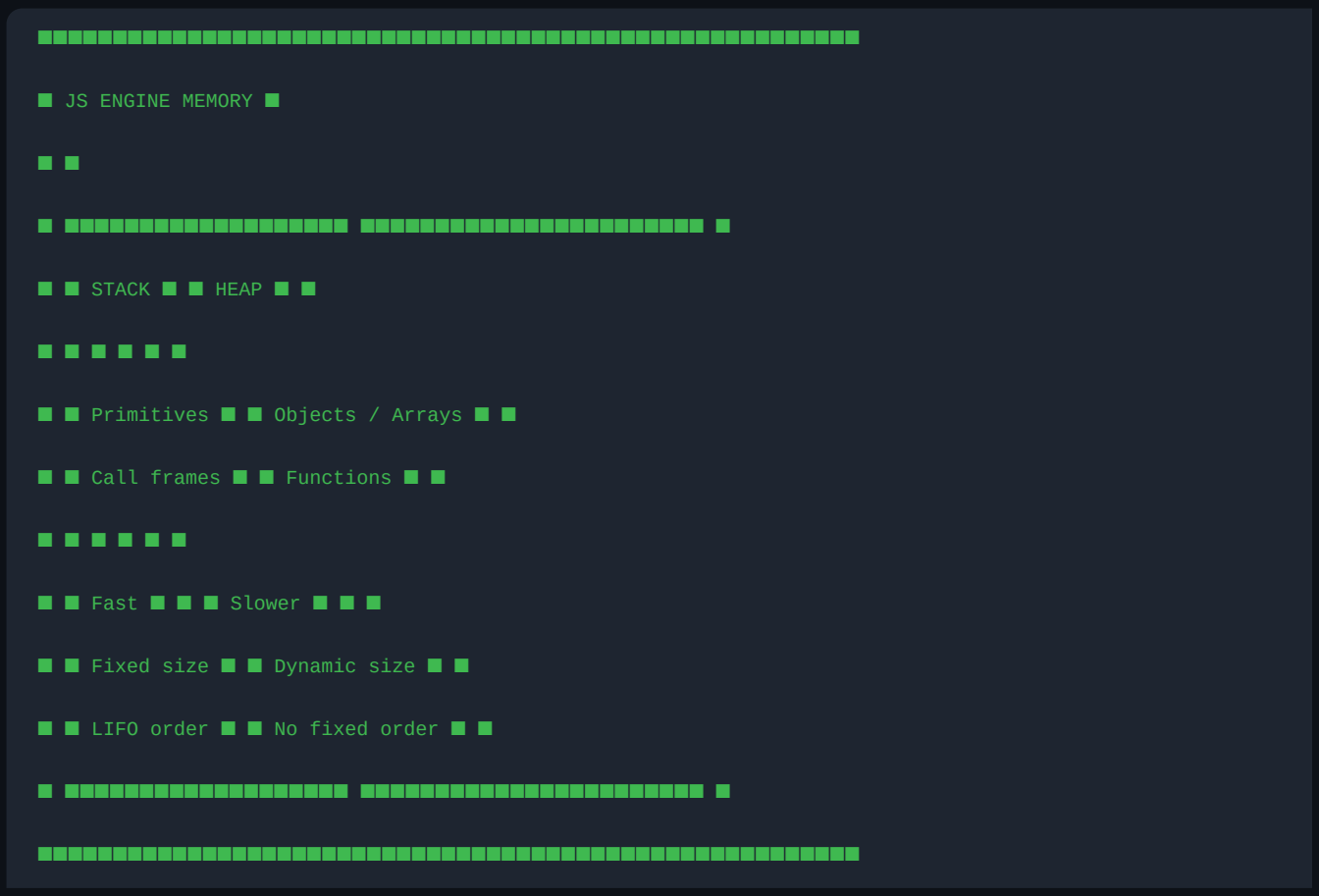
JavaScript

Stack & Heap Memory

Primitive vs Non-Primitive · Memory Internals · Interview Prep



■ STACK MEMORY — Primitives

Stack Memory — Primitives (Call by Value)

Characteristics

- ■ Fixed size, extremely fast access
- ■ Automatically managed — no Garbage Collector needed
- ■ LIFO (Last In, First Out) structure
- ■ Each variable gets its OWN independent box
- ■ Stores actual values directly

How it Works — Step by Step

```
let name = 'Akshay';

let age = 25;

let city = 'Pune';
```

```
STACK

■■■■■■■■■■■■■■■■■■■■

■ city ■ 'Pune' ■ ← pushed last (top)

■■■■■■■■■■■■■■■■■■■■

■ age ■ 25 ■

■■■■■■■■■■■■■■■■■■■■

■ name ■'Akshay'■ ← pushed first (bottom)

■■■■■■■■■■■■■■■■■■■■
```

Call by Value — Copy Behaviour

```
let a = 100;

let b = a; // b gets a FRESH COPY of a's value


b = 200; // only b changes


console.log(a); // 100 ■ — untouched

console.log(b); // 200
```

```
STEP 1: let a = 100 STEP 2: let b = a

STACK STACK

■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■
```

```
■ a ■ 100 ■ ■ b ■ 100 ■ ← NEW box

■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■

■ a ■ 100 ■ ← untouched

STEP 3: b = 200 ■■■■■■■■■■■■■■■■

STACK

■■■■■■■■■■■■■■■■

■ b ■ 200 ■ ← only b updated

■■■■■■■■■■■■■■■■

■ a ■ 100 ■ ← a still 100

■■■■■■■■■■■■■■■■
```

Stack in Function Calls

```
function greet(username) {

let message = 'Hello ' + username;

return message;

}


let user = 'Akshay';

let result = greet(user);
```

```
STACK (top = newest, bottom = oldest)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ greet() Frame ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■

■ ■ username ■ 'Akshay' ■ ■ ← copy of user

■ ■ message ■ 'Hello Akshay' ■ ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ Global Frame ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■

■ ■ user ■ 'Akshay' ■ ■

■ ■ result ■ (waiting...) ■ ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■


After greet() returns → its frame POPPED off stack
```

■ HEAP MEMORY — Non-Primitives

Heap Memory — Non-Primitives (Call by Reference)

Characteristics

- ■ Dynamic size — grows and shrinks at runtime
- ■ Managed by the Garbage Collector
- ■ Stores actual object/array data
- ■ Stack variable holds only the REFERENCE (pointer/address)
- ■ Multiple variables can point to the SAME heap location

How it Works — Step by Step

```
let user = { name: 'Akshay', age: 25 };
```

```
STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■

■ user ■ 0xFF4A2C ■■■■■■■■■■■ { name: 'Akshay', ■

■■■■■■■■■■■■■■■■■■■■ ■ age: 25 } ■

holds address ■■■■■■■■■■■■■■■■■■■■■■■■■

(reference/pointer) actual data lives here
```

Call by Reference — The Trap ■

```
let obj1 = { name: 'Akshay' };

let obj2 = obj1; // copies the REFERENCE, not the data


obj2.name = 'Rahul';


console.log(obj1.name); // 'Rahul' ■ — BOTH changed!

console.log(obj2.name); // 'Rahul'
```

```
STEP 1: let obj1 = { name: 'Akshay' }

STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ obj1 ■ 0xFF4A2C ■■■■■■■■ {name: 'Akshay'} ■ addr: 0xFF4A2C

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■


STEP 2: let obj2 = obj1 ← copies the ADDRESS, not the object
```

```
STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ obj1 ■ 0xFF4A2C ■■■■■■■■ {name: 'Akshay'} ■ addr: 0xFF4A2C

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ obj2 ■ 0xFF4A2C ■■■■■■■■■■■■■■■▲

■■■■■■■■■■■■■■■■■■■■ SAME heap location!


STEP 3: obj2.name = 'Rahul'

STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ obj1 ■ 0xFF4A2C ■■■■■■■■ {name: 'Rahul' } ■ ← MUTATED

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ obj2 ■ 0xFF4A2C ■■■■■■■■■■■■■■■▲

■■■■■■■■■■■■■■■■■■■■ obj1 ALSO sees the change!
```

Same for Arrays

```
let arr1 = [1, 2, 3];

let arr2 = arr1; // copies reference


arr2.push(4);


console.log(arr1); // [1, 2, 3, 4] ■ — arr1 also changed!
```

```
STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■

■ arr1 ■ 0xAB1234 ■■■■■■■■ [1, 2, 3, 4]■

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■

■ arr2 ■ 0xAB1234 ■■■■■■■■■■■■■■■■▲

■■■■■■■■■■■■■■■■■■■■ same heap location!
```

■ FIXING THE REFERENCE PROBLEM

Cloning — Shallow vs Deep

Shallow Clone (1 level deep)

```
let original = { name: 'Akshay', age: 25 };


// Method 1 — Spread operator

let copy1 = { ...original };


// Method 2 — Object.assign

let copy2 = Object.assign({}, original);


copy1.name = 'Rahul';

console.log(original.name); // 'Akshay' ■ — original safe now
```

```
STACK HEAP

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■

■ original ■ ref1 ■■■■■ {name:'Akshay',age:25}■ addr: ref1

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■

■ copy1 ■ ref2 ■■■■■ {name:'Rahul', age:25}■ addr: ref2 (NEW!)

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■

different refs now!
```

■■ Shallow Clone FAILS for Nested Objects!

Spread operator only copies the first level. Nested objects still share the same heap reference!

```
let user = {

name: 'Akshay',

address: { city: 'Pune' } // nested object

};


let copy = { ...user }; // shallow clone

copy.address.city = 'Mumbai'; // modifying nested


console.log(user.address.city); // 'Mumbai' ■ — still linked!
```

```
STACK HEAP
```

```
■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ user ■ ref1 ■■■■■■ name: 'Akshay', address: ref3 ■■■ ■
■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ copy ■ ref2 ■■■■■ name: 'Akshay', address: ref3 ■■■
■■■■■■■■■■■■■■■ ▼ ■
■■■■■■■■■■■■■■■■■■■■ ■
■ {city: 'Mumbai'} ■■■■■■
■■■■■■■■■■■■■■■■■■■■
Both user and copy point to the SAME nested ref3!
```

Deep Clone — The Real Fix ■

```
// ■ Method 1 — structuredClone (modern, recommended)
let deepCopy = structuredClone(user);

// ■ Method 2 — JSON trick (loses functions / Date / undefined)
let deepCopy2 = JSON.parse(JSON.stringify(user));

deepCopy.address.city = 'Delhi';
console.log(user.address.city); // 'Pune' ■ — fully independent
```

```
STACK HEAP
■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ user ■ ref1 ■■■■■■ name:'Akshay', address:ref3 ■
■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ deep ■ ref2 ■■■■■■ name:'Akshay', address:ref4 ■ ← brand new copy
■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
↓ ↓
■■■■■■■■■■■■ ■■■■■■■■■■■■■
■{city:Pune}■ ■{city:Delhi}■ ← fully separate!
■■■■■■■■■■■■ ■■■■■■■■■■■■■
```

■ GARBAGE COLLECTION

Garbage Collection — Heap Cleanup

When no variable in the Stack holds a reference to a Heap object, the GC marks it as unreachable and frees the memory automatically.

```
let user = { name: 'Akshay' }; // heap allocated

user = null; // reference removed from stack

// GC sees no variable pointing to that heap object

// → marks it for cleanup → memory freed ■
```

```
BEFORE user = null: AFTER user = null:

STACK HEAP STACK HEAP

■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■

■ user ■ref1■■■{name:'Akshay'}■ ■ user ■ null ■ ■{name:'Akshay'}■

■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■

orphaned! GC will clean ↑
```

■ COMPLETE COMPARISON

Stack vs Heap — Full Comparison

Feature	Stack	Heap
Stores	Primitives, references	Actual objects/arrays
Size	Fixed, limited	Dynamic, large
Access speed	Very fast ■	Slower ■
Management	Auto (scope-based)	Garbage Collector
Data types	string, number, bool, null, undefined, symbol, bigint	object, array, function
Copy behaviour	Value copied	Reference copied
Cleared when	Function scope ends	GC runs (no more refs)

typeof Cheat Sheet — Must Memorise ■

```
typeof "hello" // "string"

typeof 42 // "number"

typeof true // "boolean"

typeof undefined // "undefined"

typeof null // "object" ← BUG! (historical)

typeof Symbol() // "symbol"

typeof 42n // "bigint"

typeof {} // "object"

typeof [] // "object" ← arrays are objects!

typeof function(){} // "function" ← special case


// ■ Proper array check:

Array.isArray([]); // true
```

■ INTERVIEW Q & A

Top Interview Questions

Q: Where are primitives and objects stored in memory?

→ Primitives are stored directly in Stack memory. Objects/Arrays are stored in Heap memory, and the Stack holds only a reference (pointer) to the heap location.

Q: Why does modifying a copied object affect the original?

→ Because only the reference is copied, not the actual data. Both variables point to the same heap memory location, so changes through either variable affect the same object.

Q: What is the difference between shallow clone and deep clone?

→ Shallow clone copies only the first level — nested objects still share the same heap reference. Deep clone using structuredClone() creates a completely independent copy at all levels.

Q: What happens to heap memory when a variable is set to null?

→ The stack reference is removed. If no other variable holds a reference to that heap object, the Garbage Collector identifies it as unreachable and frees the memory.

Q: Are functions stored in Stack or Heap?

→ Function definitions are stored in Heap. The function call frame (execution context — local variables, arguments) is pushed onto the Stack and popped when the function returns.

Q: What is the difference between == and === regarding references?

→ Both compare objects by reference, not by value. {} === {} is false because they are different heap locations. const a = {}; const b = a; a === b is true — same reference.

■ Golden Rule

Primitive → Stack → Copy of VALUE → Original SAFE ■

Reference → Heap → Copy of ADDRESS → Original at RISK ■■

(use structuredClone() to be safe)